# StudentClassifier — Student Dropout & Academic Success Classification

*Machine Learning Assignment 2 — Submission Document*

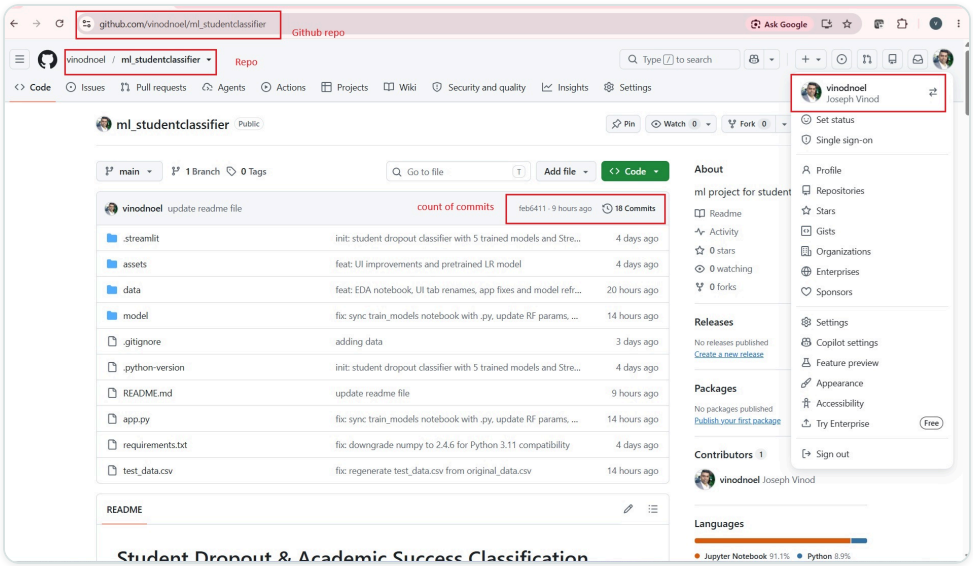
| | |
|---|---|
| Name | **Joseph Mruthunjaya Vinod Noel** |
| BITS ID | **2025AC05003** |
| Task | **Comparison of five classification algorithms on the UCI Student Dropout & Academic Success dataset** |
| Deliverables | **GitHub repository · Live Streamlit app · BITS Virtual Lab execution screenshots · README** |

## 1. GitHub Repository Link

```
https://github.com/vinodnoel/ml_studentclassifier
```

The repository is public and contains:

- **Complete source code** — `app.py` (Streamlit entry point) implementing all six tabs of the app.
- `requirements.txt` — all Python dependencies needed to reproduce the environment and deploy the app.
- **README.md** — problem statement, dataset description, models used, metric comparison table and per-model observations. Reproduced in full in Section 4.3 of this document.
- **Test data (CSV)** — `test_data.csv`, plus `data/` holding the full UCI dataset (4,424 rows, 37 columns including the target).
- `model/` — the training notebook and five serialized `.pkl` models (Logistic Regression, Decision Tree, kNN, Naive Bayes, Random Forest) plus `metrics.json` and `schema.json`.




The public GitHub repository **vinodnoel/ml_studentclassifier**, showing the full file/folder listing (`.streamlit/`, `assets/`, `data/`, `model/`, `app.py`, `requirements.txt`, `test_data.csv`), 18 commits, and the signed-in GitHub account confirming ownership of the repository.

## 2. Live Streamlit App Link

```
https://studentclassifier-josephvinod-bits.streamlit.app/
```

Deployed on **Streamlit Community Cloud** from the `main` branch of the repository above. Clicking the link opens the interactive frontend directly, with six tabs:

- **Dataset** — class balance and per-feature distributions by outcome class.
- **Train a Model** — select one model, tune its hyperparameters, and view all six evaluation metrics, a confusion matrix, classification report, and feature importances/coefficients.
- **Compare Models** — train all five models with default hyperparameters on the same split and compare them side by side.
- **Model Report** — deep-dive diagnostics for any pre-trained model: confusion matrix, classification report, and one-vs-rest ROC curves.
- **Predict** — draw a random test sample and view the model's prediction with per-class probabilities and top feature values.
- **Upload Data** — upload your own test CSV, matched against the documented column schema, to drive every other tab.

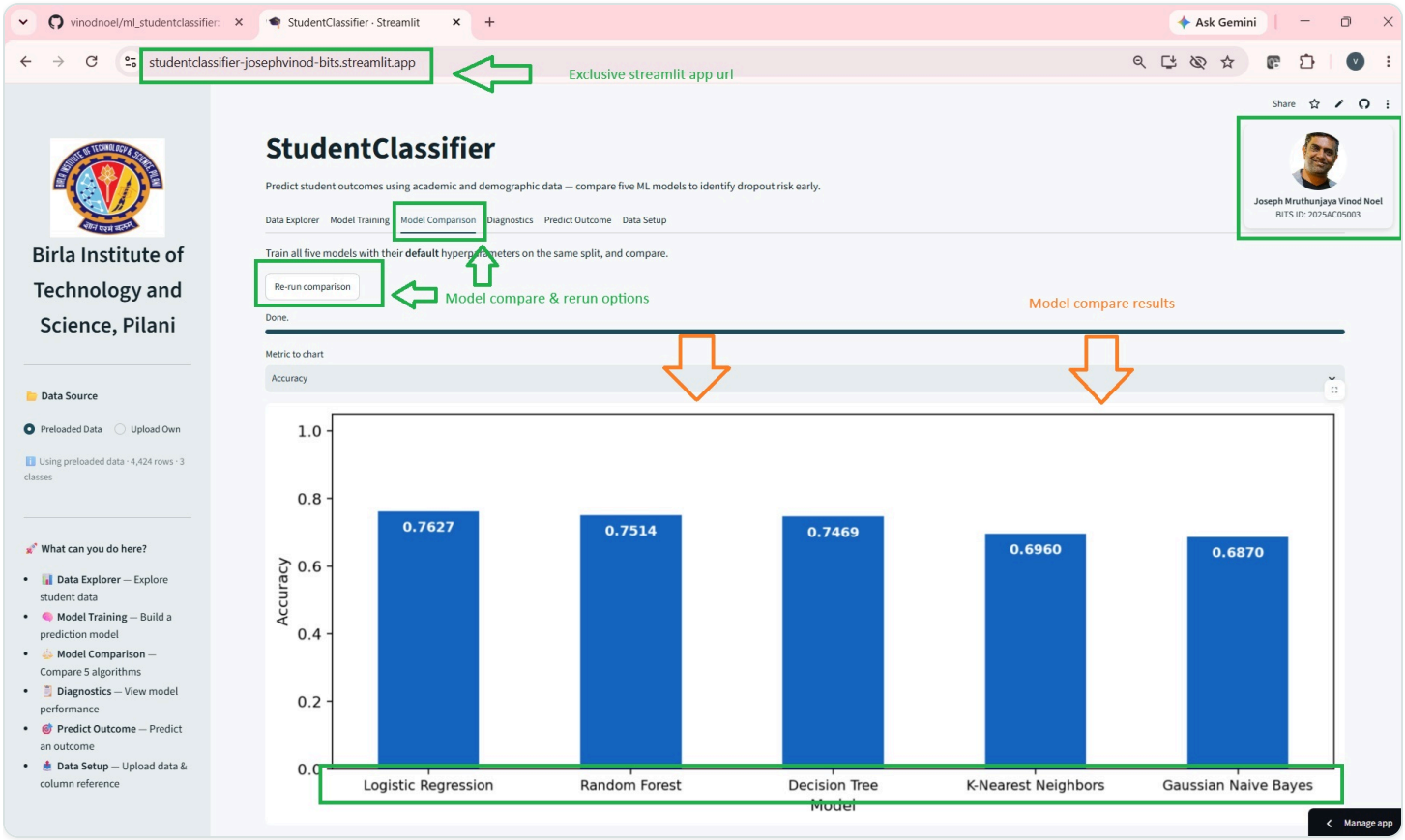



**Figure 0** — The deployed **StudentClassifier** app, Compare Models tab: all five classifiers trained on the same split and charted by accuracy (Logistic Regression highest at 0.7627), with the live Streamlit Cloud URL and the BITS identification badge (name & ID) visible top-right.

## 3. Screenshot

### Part A: Assignment Execution on BITS Virtual Lab

> **Lab verification.** The screenshots below were captured on the BITS Virtual Lab machine, accessed via `argo-rdp.codeargo.net`. The shell prompt `cloud@2025ac05003` — the lab hostname, which matches the BITS ID **2025AC05003** — confirms the commands were run on the lab environment and not on a local machine.

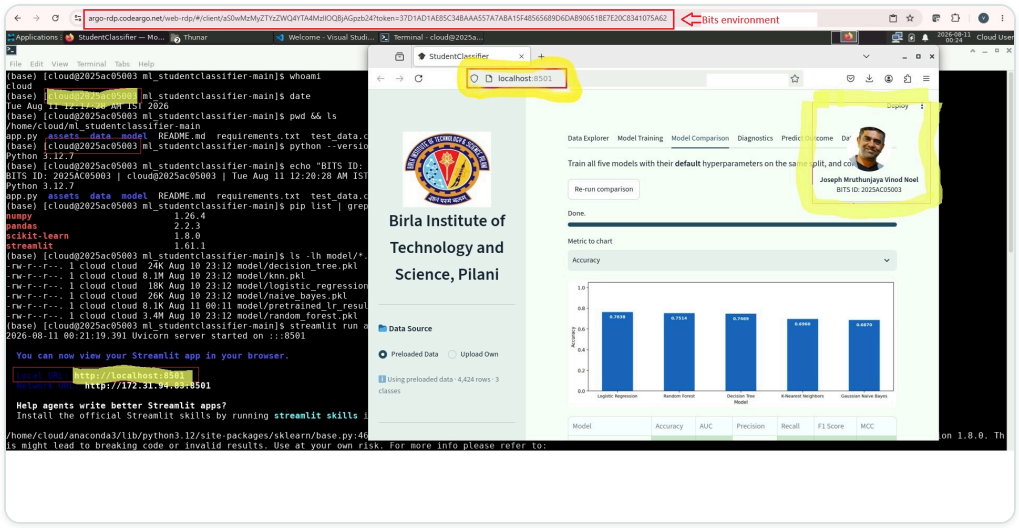



**Figure 1** — BITS Virtual Lab environment, viewed via `argo-rdp.codeargo.net`: the terminal (left) confirms the lab hostname `cloud@2025ac05003`, while the app's Compare Models tab (right) is already running at `localhost:8501` with the BITS identification badge visible top-right.

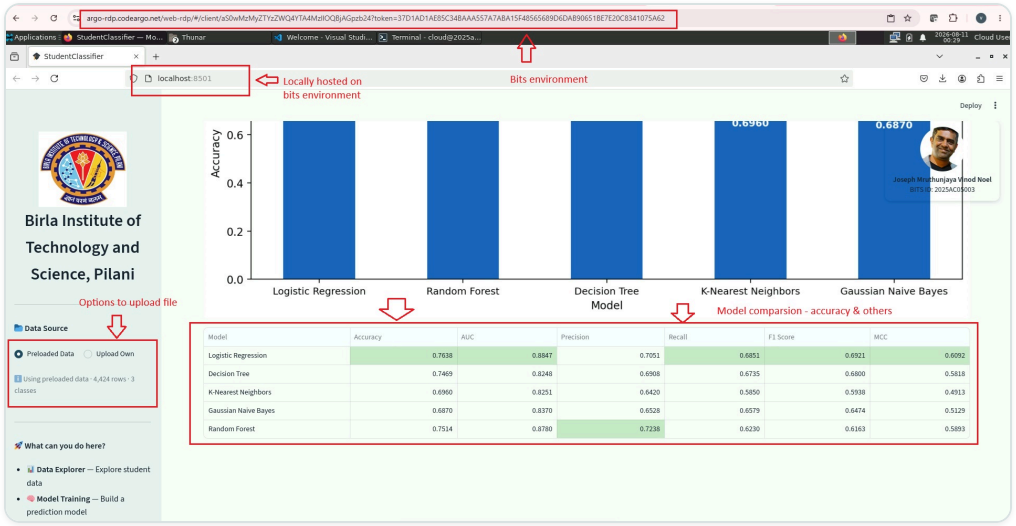



**Figure 2** — App running live on the BITS Virtual Lab, viewed full-screen in the lab's browser at `localhost:8501` (Compare Models tab): the metrics table for all five models — Accuracy, AUC, Precision, Recall, F1 Score, MCC — with the Data Source panel visible in the sidebar. The `argo-rdp.codeargo.net` address bar confirms this is the remote lab session.

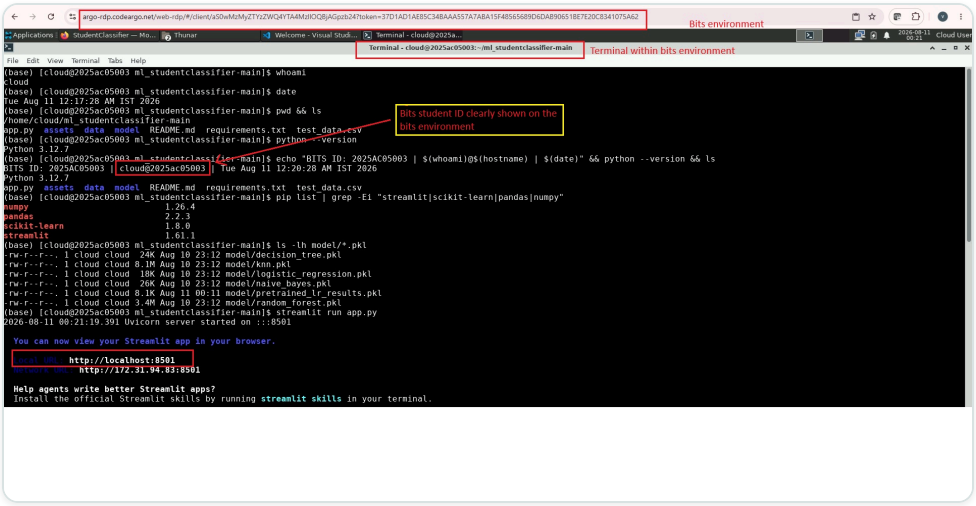



**Figure 3** — BITS Virtual Lab terminal: environment check (`whoami`, `date`, `pwd` / `ls`, `python --version`, `pip list` showing numpy/pandas/scikit-learn/streamlit versions, and the five saved `model/*.pkl` files) and `streamlit run app.py` starting the server on `localhost:8501`, confirming host `cloud@2025ac05003` matches BITS ID 2025AC05003.

## Part B: Screenshots — App Walkthrough (Live Deployed App)

The screenshots below step through the remaining tabs of the deployed app at `studentclassifier-josephvinod-bits.streamlit.app`. The BITS identification badge (name & BITS ID) is visible in the top-right corner of every tab, and the browser address bar confirms the live Streamlit Cloud URL.

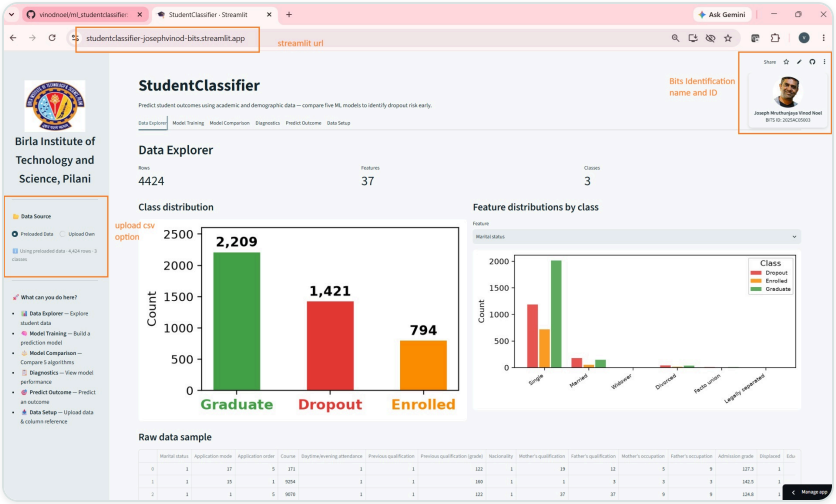



**Figure 4** — Dataset tab: data profile (4,424 rows · 37 features · 3 classes), class-distribution bar chart (Graduate 2,209 · Dropout 1,421 · Enrolled 794), feature distributions by class, and a raw data sample table.

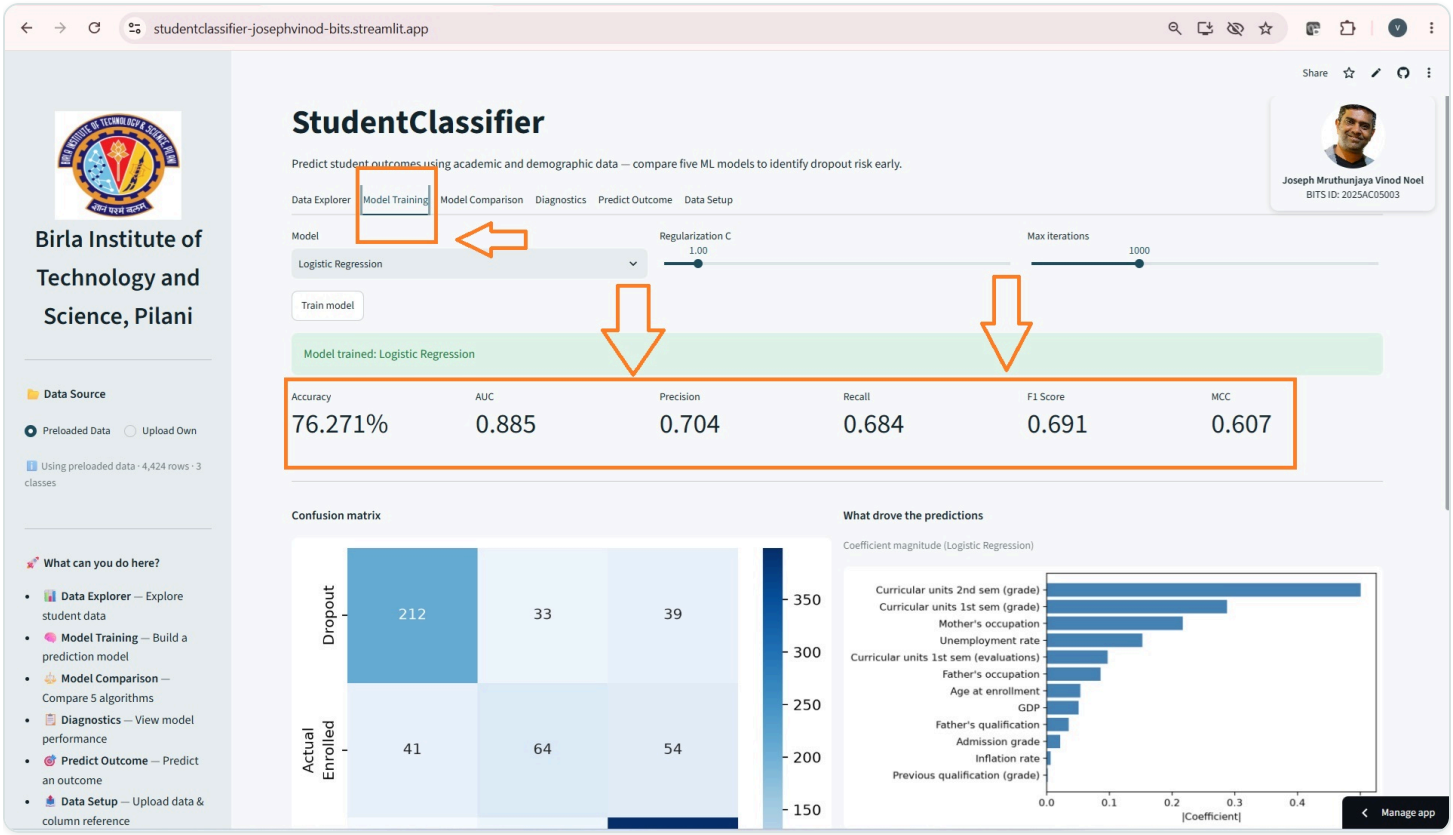



**Figure 5** — Train a Model tab: Logistic Regression trained interactively with tunable Regularization C and Max Iterations, reporting Accuracy (76.271%), AUC (0.885), Precision, Recall, F1 and MCC, a confusion matrix, and a coefficient-magnitude chart of what drove the predictions.

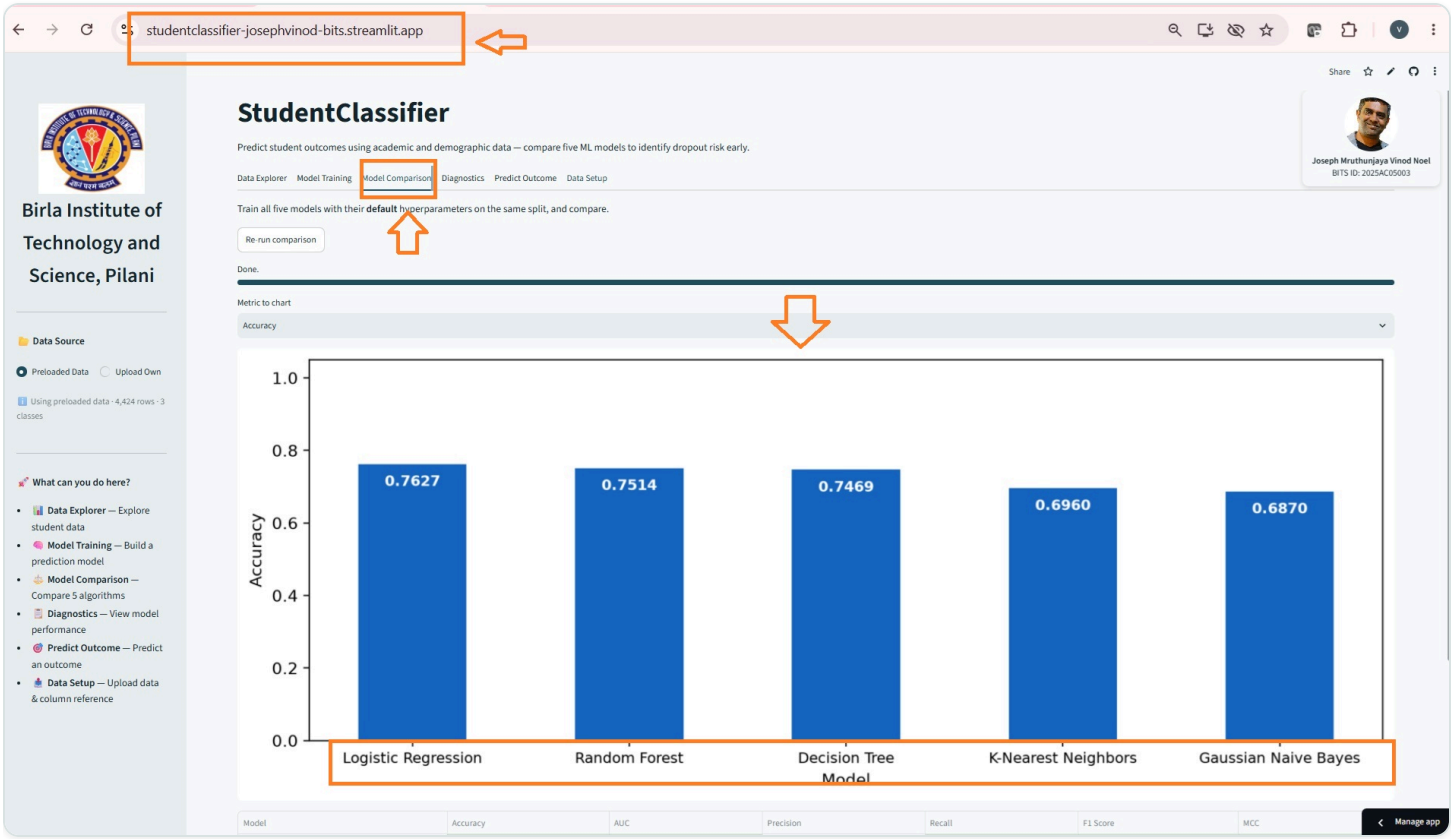



**Figure 6** — Model Comparison tab: all five models — Logistic Regression, Random Forest, Decision Tree, K-Nearest Neighbors, Gaussian Naive Bayes — trained with their default hyperparameters on the same split and compared by accuracy in a single bar chart, with a Re-run comparison control.

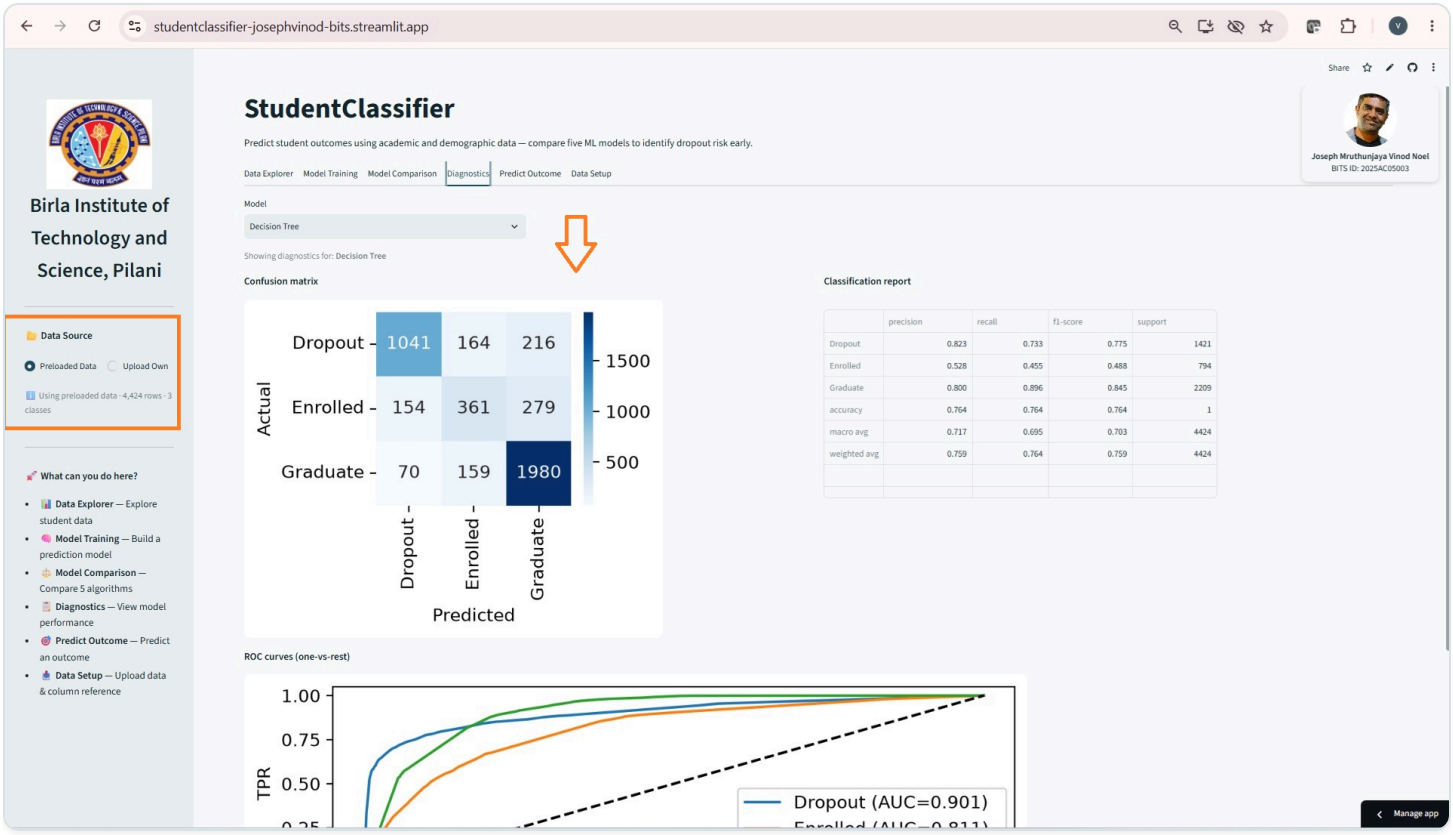



**Figure 7** — Diagnostics tab: deep-dive for the Decision Tree model — confusion matrix heatmap (Dropout / Enrolled / Graduate), a full per-class classification report (precision, recall, f1-score, support), and one-vs-rest ROC curves with AUC per class.

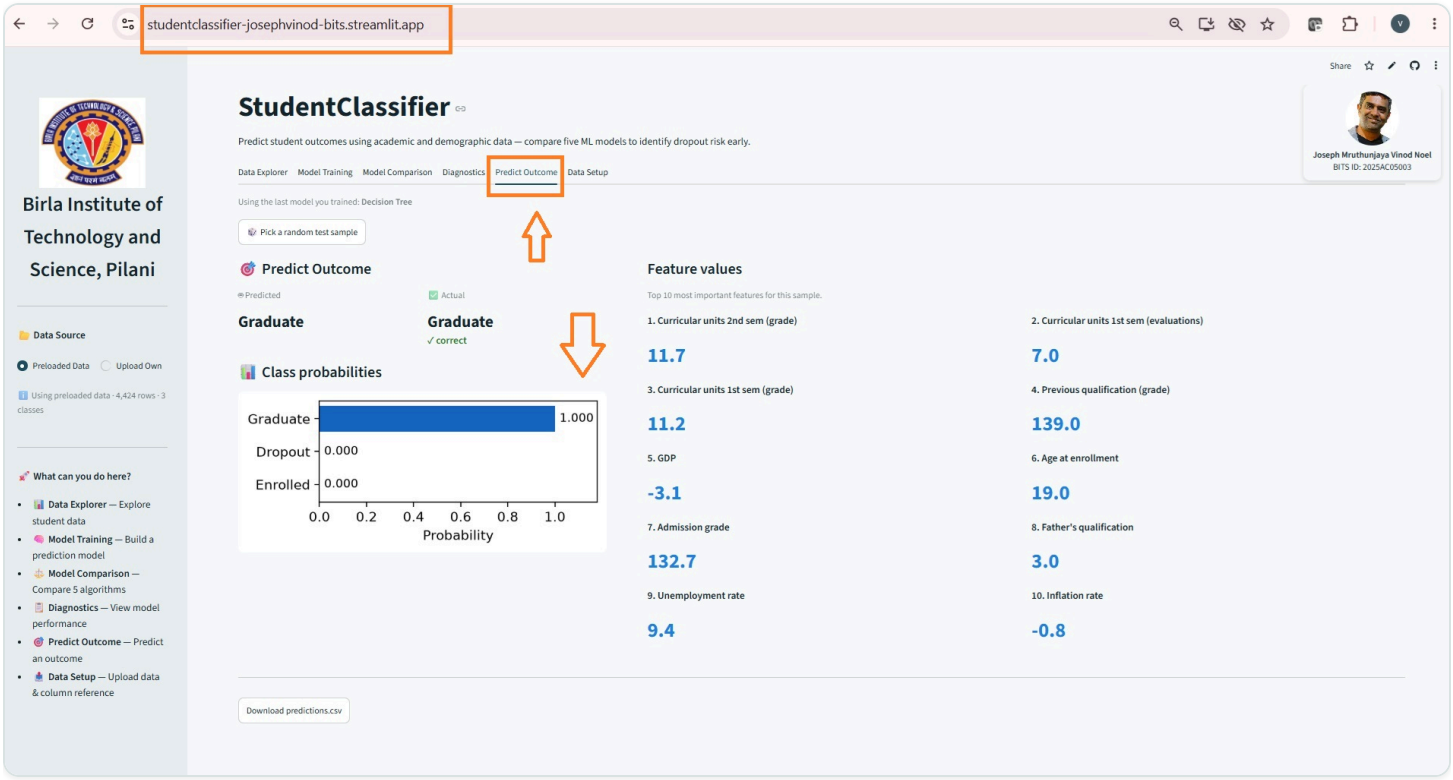



**Figure 8** — Predict Outcome tab: a random test sample is drawn and classified (predicted Graduate, actual Graduate — correct), with the class-probability breakdown and the top 10 feature values driving the prediction.

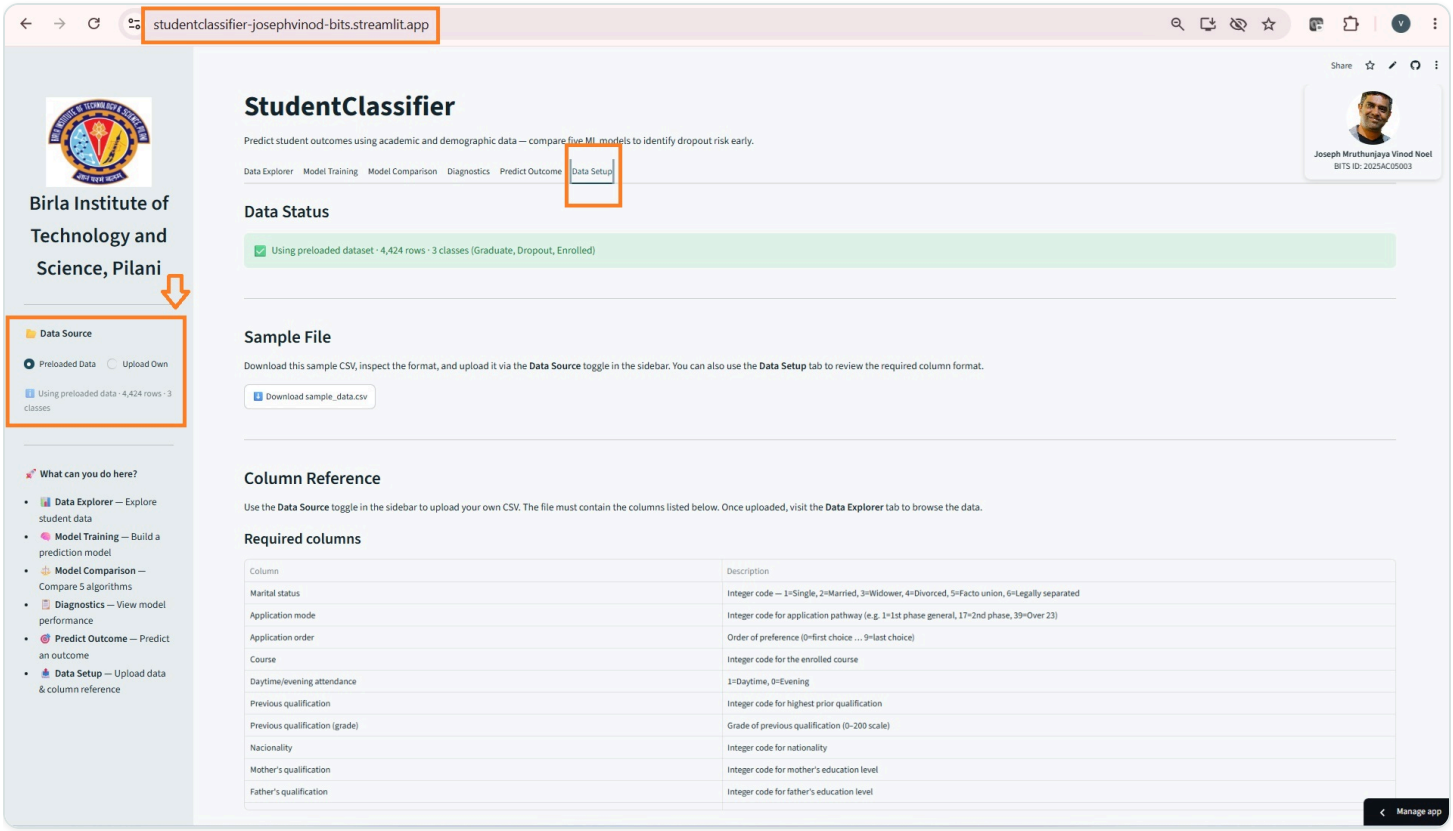



**Figure 4.6** — Data Setup tab: switching the sidebar's Data Source toggle to **Upload Own** (1) lets the user upload their own test CSV; the Data Status panel automatically validates the file and confirms the loaded rows and class labels (2), alongside a downloadable sample CSV and the full column reference table.

# 4. GitHub README Content

Created by Joseph Mruthunjaya Vinod Noel · BITS ID: 2025AC05003

## 4.1 Project Folder Structure

The repository's folder structure, shown both in the local VS Code Explorer and on GitHub itself:

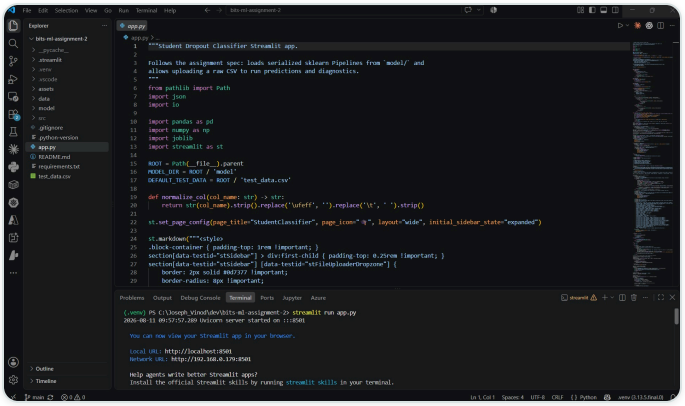


**Figure 9** — Project folder structure in VS Code (local development environment): `assets/`, `data/`, `model/`, `src/`, `.streamlit/`, `app.py`, `README.md`, `requirements.txt`, `test_data.csv` — with `app.py` open in the editor and the integrated terminal confirming `streamlit run` starts the server at `localhost:8501`.

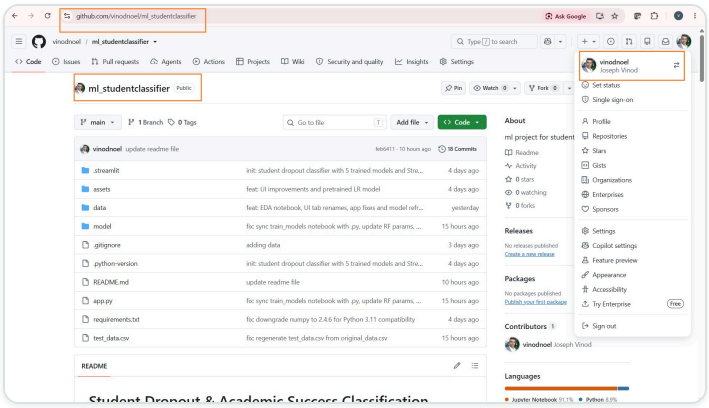


**Figure 10** — The same project folder structure on the public GitHub repository **vinodnoel/ml_studentclassifier** at `github.com/vinodnoel/ml_studentclassifier` — with commit history (18 commits) and the rendered README.

**Project Structure**

```
ml_studentclassifier/
├── app.py
├── requirements.txt
├── README.md
├── test_data.csv
├── data
├── model/
|   ├── train_models.ipynb
|   ├── logistic_regression.pkl
|   ├── decision_tree.pkl
|   ├── knn.pkl
|   ├── naive_bayes.pkl
|   ├── random_forest.pkl
|   ├── metrics.json
|   └── schema.json
└── .streamlit/
    └── config.toml
```

**Figure 10.1** — Project Structure, as documented in the README: the full repository layout — `app.py`, `requirements.txt`, `README.md`, `test_data.csv`, `data/`, `model/` (training notebook, five `.pkl` models, `metrics.json`, `schema.json`), and `.streamlit/config.toml`.

### 4.2 requirements.txt

All Python dependencies pinned for a reproducible environment and deployment:

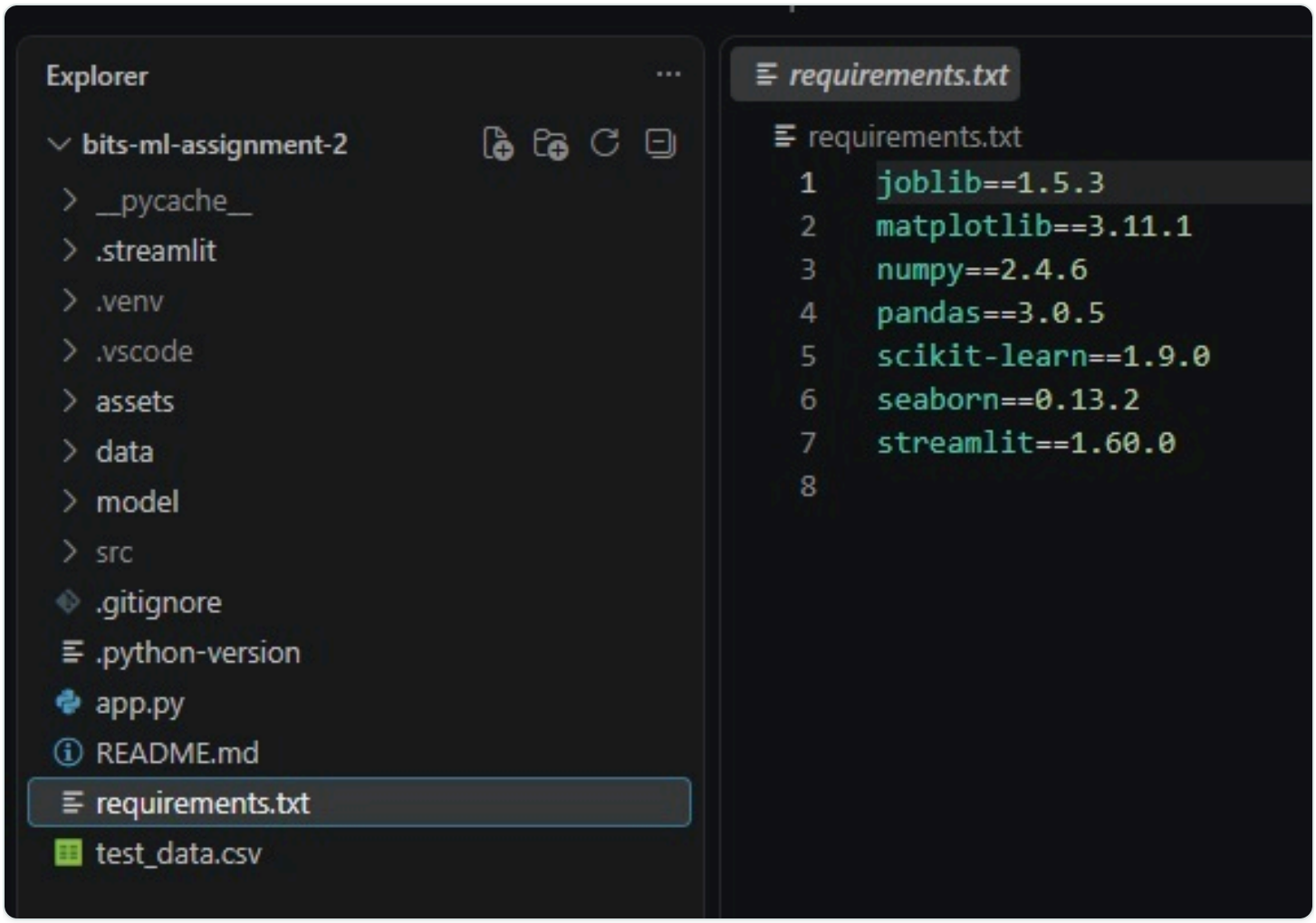



**Figure 11** — `requirements.txt` open in VS Code: `joblib`, `matplotlib`, `numpy`, `pandas`, `scikit-learn`, `seaborn`, `streamlit`, each pinned to a specific version.

### 4.3 README.md Content

**a. Problem Statement**

Predicting whether a student will drop out, remain enrolled, or graduate is normally identified only after it is too late to intervene. This project builds and compares five classic supervised classification algorithms that predict a student's academic outcome (one of 3 classes) from 36 demographic, academic, and socioeconomic features recorded at enrolment time, and packages them in an interactive Streamlit app for training, evaluating, and testing the models.

**b. Dataset Description**

- Source: [UCI ML Repository — Predict Students' Dropout and Academic Success (Dataset #697)]
- Samples: 4,424 student records
- Classes (3): Dropout, Enrolled, Graduate
- Features (36): demographic background (age, gender, nationality, marital status, international student flag), academic performance (previous qualification grade, admission grade, curricular units credited/enrolled/approved/graded across two semesters), and socioeconomic context (parents' education and occupation, scholarship holder, debtor, tuition fees up to date, unemployment rate, inflation rate, GDP).

- Split: 75% train (3,318) / 25% test (1,106), stratified by class, with `random_state=42`.
- Preprocessing: Median imputation + standard scaling for 12 continuous numeric features; mode imputation + one-hot encoding for 24 integer-coded nominal/categorical features.
- Class balance: Moderately imbalanced — Graduate (49.9%) dominates; Enrolled (17.9%) is the smallest class and hardest to predict, which depresses macro-averaged recall across all models.
- Meets assignment minimums: 36 features (≥ 12 required) and 4,424 instances (≥ 500 required).

*Citation: Realinho, V., Vieira Martins, M., Machado, J., & Baptista, L. (2022). Predict Students' Dropout and Academic Success. UCI Machine Learning Repository. https://doi.org/10.24432/C5MC89*

### Exploratory Data Analysis

A full EDA notebook is available at `model/eda_student_dropout.ipynb`. Key findings:

- **Graduate students dominate (~49%)** with Enrolled the smallest class (~18%), creating the moderate imbalance that suppresses macro-averaged recall across all models.
- **Admission grade and 2nd-semester grade are the strongest numeric separators** — Graduate students score noticeably higher on both, explaining their prominence as top Logistic Regression features.
- **Tuition fees up to date and scholarship status are the most discriminating categorical features** — students current on fees and scholarship holders have markedly higher Graduate proportions and lower Dropout rates.
- **Exact class counts:** Dropout 1,421 (32.1%), Enrolled 794 (17.9%), Graduate 2,209 (49.9%).
- **Column-typing audit confirms the preprocessing split is semantically correct**, not just dtype-based: 24 integer-coded columns with ≤30 unique values are genuine nominal categories (e.g. Course, Marital status), and the 12 remaining columns are truly continuous (grades, rates, GDP).
- **Zero missing values and zero duplicate rows** in the raw dataframe — the pipeline's `SimpleImputer` never actually fires on this dataset and is retained purely as a safeguard for user-uploaded data in the Streamlit app.
- **Numeric feature ranges span up to 43×** (e.g. GDP ≈ [-4, 3] vs. Father's occupation ≈ [0, 195]), confirming `StandardScaler` is necessary rather than optional for distance- and gradient-based models.
- **IQR outlier audit flags two columns** for elevated outlier rates — Age at enrollment (10.0% of rows) and 1st-semester grade (16.4%) — both retained as domain-plausible (older re-entrant students, zero-grade non-completions) rather than data errors.
- **Correlation heatmap shows admission grade, 1st-semester grade, and 2nd-semester grade are moderately positively correlated**, introducing mild multicollinearity that Logistic Regression handles via standardisation but that contributes to the single Decision Tree's instability.
- A full **data dictionary** covering all 36 features + target (sourced from the UCI variable table, CC BY 4.0) is included in the notebook, since 24 of 36 features are integer-coded categoricals that are unreadable without a legend.

### c. GitHub Repository Link

https://github.com/vinodnoel/ml_studentclassifier

### Live Streamlit App Link

https://studentclassifier-josephvinod-bits.streamlit.app/

### d. Models Used

Five classic classification algorithms were trained on the same train/test split using consistent preprocessing:

1. Logistic Regression
2. Decision Tree Classifier
3. K-Nearest Neighbors (kNN)
4. Gaussian Naive Bayes
5. Random Forest (Ensemble)

### Comparison Table

| ML Model Name | Accuracy | AUC | Precision | Recall | F1 | MCC |
|---|---|---|---|---|---|---|
| **Logistic Regression** | **0.7613** | **0.8812** | 0.7122 | **0.6856** | **0.6949** | **0.6046** |
| **Decision Tree** | 0.7306 | 0.8203 | 0.6734 | 0.6538 | 0.6606 | 0.5547 |
| **K-Nearest Neighbors** | 0.6899 | 0.8202 | 0.6415 | 0.5841 | 0.5943 | 0.4814 |
| **Gaussian Naive Bayes** | 0.6763 | 0.8340 | 0.6503 | 0.6554 | 0.6409 | 0.5025 |
| **Random Forest (Ensemble)** | 0.7459 | 0.8735 | **0.7129** | 0.6192 | 0.6146 | 0.5791 |

Precision, Recall, and F1 are macro-averaged across all 3 classes; AUC is one-vs-rest macro-averaged. Evaluated on the held-out test set (1,106 samples, test_size=0.25, random_state=42).

### Observations

| | |
|---|---|
| **Logistic Regression** | Highest accuracy (0.7613) and F1 (0.6950) of all five models. The linear decision boundary works well here because, after standardisation and one-hot expansion, the Dropout and Graduate classes are broadly separable in feature space — particularly along the curricular-unit grade and approval-rate axes. The Enrolled class, which sits ambiguously between the other two outcomes, is the main source of misclassification. Coefficient magnitudes also provide direct interpretability: tuition fees up to date and 2nd-semester approved units emerge as the strongest predictors. |
| **Decision Tree** | Second-lowest AUC (0.8203). The single tree's recursive splits can capture non-linear feature interactions (e.g. age × scholarship status), but depth-capping at `max_depth=8` to prevent overfitting leaves some boundary complexity unexplored. Without ensemble averaging, predictions near split thresholds are noisy, resulting in a meaningful accuracy gap (0.7306) vs. Logistic Regression — despite both models sharing identical preprocessing. |

| | |
|---|---|
| **K-Nearest Neighbors** | Weakest F1 (0.5943) of all five models, and second-weakest accuracy (0.6899) — Gaussian Naive Bayes has the single lowest accuracy at 0.6763. One-hot encoding the 24 nominal columns inflates the feature space, diluting Euclidean distances between samples — the classic curse of dimensionality for kNN. The Enrolled minority class suffers most, pulling macro recall down to 0.5841. With `k=15`, the model is also forced to average across many potentially dissimilar neighbours in this high-dimensional space. |
| **Gaussian Naive Bayes** | Third-best AUC (0.8340) and the lowest accuracy (0.6763) of all five models. Gaussian Naive Bayes assumes each feature follows a class-conditional Gaussian distribution. With `var_smoothing=1e-2` a fraction of the largest feature variance is added to all per-class variances, preventing near-zero variance on low-variance integer columns. The independence assumption is only approximately satisfied — several curricular-unit features are correlated — but the calibrated probability estimates still produce a useful AUC, and its F1 (0.6409) actually edges out both Random Forest (0.6146) and KNN (0.5943). |
| **Random Forest (Ensemble)** | Deliberately bounded to `n_estimators=150, max_depth=12, min_samples_leaf=5` (down from an initial unbounded 300-tree configuration) after the larger forest produced a 22.8 MB pickle that triggered `MemoryError` crashes on Streamlit Community Cloud's free-tier container. This trades some predictive performance for deployability: accuracy drops to 0.7459 and AUC to 0.8735 — both now behind Logistic Regression — though Random Forest keeps a narrow precision edge (0.7129 vs 0.7122). Random feature subsampling still handles the collinear semester-grade columns better than a single greedy split, but with fewer, shallower trees the ensemble's variance-reduction benefit is smaller than an unconstrained forest would give. |
| **Overall Winner for your dataset?** | **Logistic Regression** — it now leads on 5 of 6 metrics (accuracy, AUC, recall, F1, and MCC), including MCC (0.6046), the metric most robust to class imbalance and generally recommended for multi-class problems like this one. Random Forest's only remaining edge is precision, by a margin of 0.0007 — well within noise. This gap widened after Random Forest's tree count and depth were deliberately constrained to keep the deployed pickle small enough for Streamlit Community Cloud's memory limit (see observation above); an unconstrained forest would likely have stayed closer to Logistic Regression, but the smaller, deployable configuration is what's actually running in production, and that is the fair basis for comparison. |

### Project Structure

```
ml_studentclassifier/
├── app.py
├── requirements.txt
├── README.md
├── test_data.csv
├── data
├── model/
│   ├── train_models.ipynb
│   ├── eda_student_dropout.ipynb
│   ├── logistic_regression.pkl
│   ├── decision_tree.pkl
│   ├── knn.pkl
│   ├── naive_bayes.pkl
│   ├── random_forest.pkl
│   ├── metrics.json
│   └── schema.json
└── .streamlit/
    └── config.toml
```

### The Streamlit App

The app has the following sections:

1. 📊 **Data Explorer** — examine the dataset: class balance bar chart and per-feature distributions by class (bar charts with human-readable labels for categorical features; KDE plots for continuous features).

2. 🧠 **Model Training** — select one of the five models, tune its hyperparameters, train on the uploaded data, and view all 6 evaluation metrics, a confusion matrix, classification report, and feature importance / coefficients.
3. ⚖️ **Model Comparison** — automatically trains all five models with default hyperparameters on the same split and displays a side-by-side metrics table (best value highlighted) and a metric bar chart.
4. 📋 **Diagnostics** — deep-dive diagnostics for any pre-trained model: confusion matrix heatmap, full classification report, and one-vs-rest ROC curves.
5. 🎯 **Predict Outcome** — pick a random sample from the uploaded test data and see the model's predicted class, true label, and per-class probability breakdown.
6. 📤 **Data Setup** — upload your own test dataset (CSV), download the sample template, and review the required column reference.

### How to Run Locally

```
git clone https://github.com/vinodnoel/ml_studentclassifier.git
cd ml_studentclassifier
python -m venv .venv
# Windows:
.venv\Scripts\activate
pip install -r requirements.txt
streamlit run app.py
```

Upload test_data.csv from the repository when prompted. The app opens at http://localhost:8501.

### Deployed on Streamlit Community Cloud

- Repository: vinodnoel/ml_studentclassifier
- Branch: main
- Main file path: app.py
- Live app: https://studentclassifier-josephvinod-bits.streamlit.app/

Any future git push to main auto-redeploys the app. No paid tier or credit card is needed for a public app on Community Cloud.

### Dataset Citation

Realinho, V., Vieira Martins, M., Machado, J., & Baptista, L. (2022). Predict Students' Dropout and Academic Success. UCI Machine Learning Repository. https://doi.org/10.24432/C5MC89

### Note on Model Count

The assignment brief states "all the 6 ML models" but enumerates exactly 5. This submission implements all 5 enumerated models: Logistic Regression, Decision Tree, K-Nearest Neighbors, Gaussian Naive Bayes, and Random Forest.

## 5. The Streamlit App

Key features of the deployed app, demonstrated below with screenshots of each in action.

### 5.a Dataset Upload Option (CSV)

The Data Setup tab lets the user replace the preloaded dataset with their own test CSV via the **Upload Own** option in the sidebar Data Source panel. Uploaded files are validated against the documented column schema and then drive every other tab (Dataset, Train a Model, Compare Models, Diagnostics, Predict Outcome).

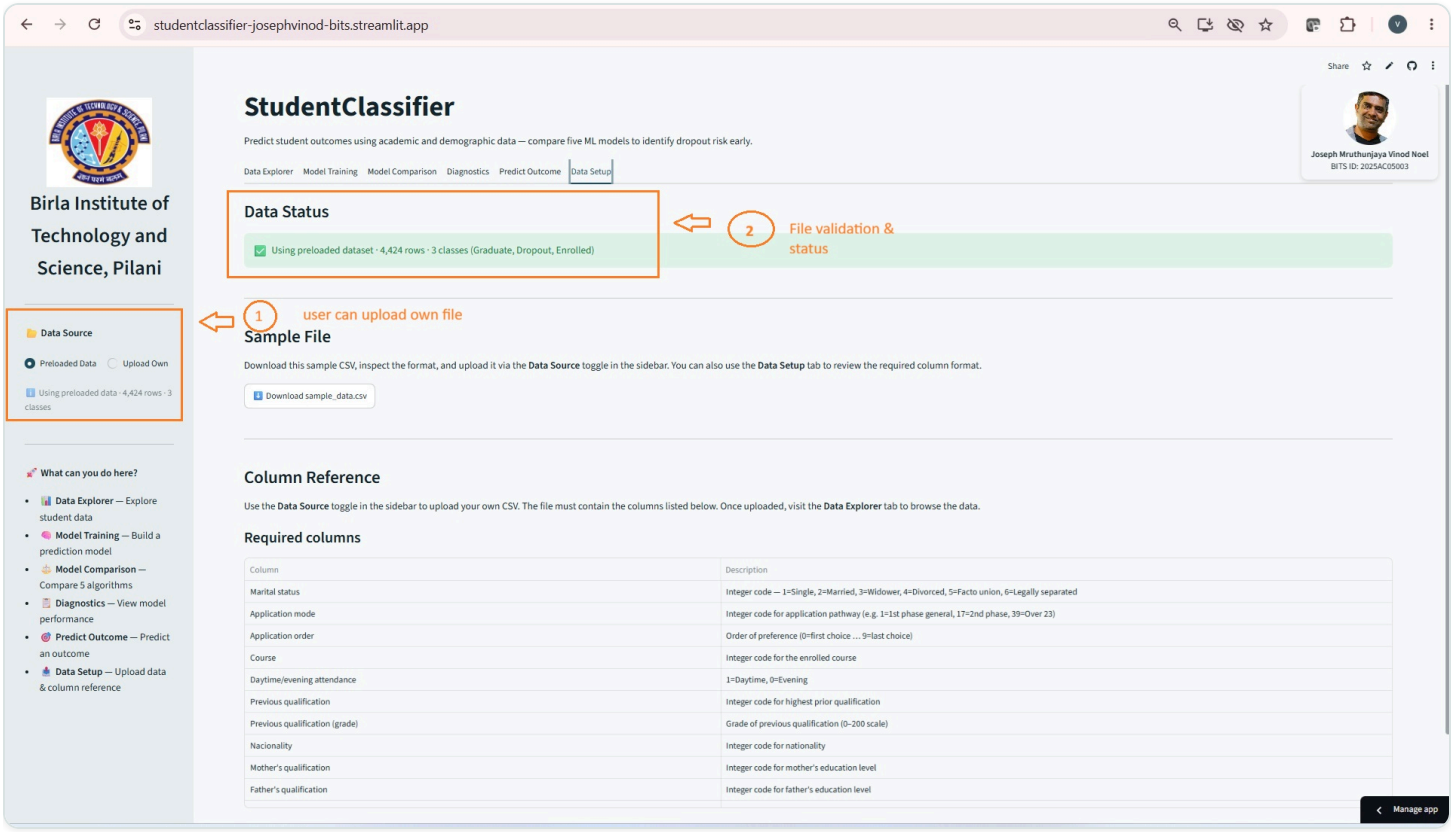



**Figure 12** — Data Setup tab: switching the sidebar's Data Source toggle to **Upload Own** (1) lets a user upload their own test CSV; the Data Status panel automatically validates and confirms the loaded rows and class labels (2).

## 5.b Model Selection Dropdown

The Train a Model tab exposes a dropdown listing all five implemented algorithms. Selecting one reveals its tunable hyperparameters (e.g. Regularization C and Max Iterations for Logistic Regression) before training.

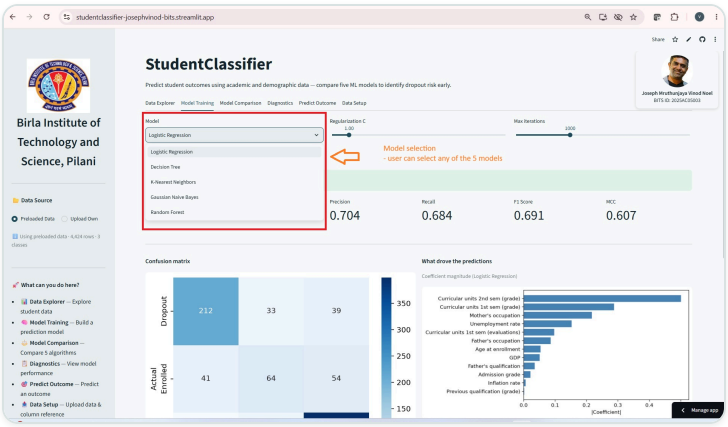



**Figure 13** — Train a Model tab: the Model dropdown open, listing all five algorithms — Logistic Regression, Decision Tree, K-Nearest Neighbors, Gaussian Naive Bayes, Random Forest.

## 5.c Display of Evaluation Metrics

Once a model is trained, the app displays all six evaluation metrics — Accuracy, AUC, Precision, Recall, F1 Score and MCC. In Compare Models, the same six metrics are shown side by side for all five models, with the best value per metric highlighted.

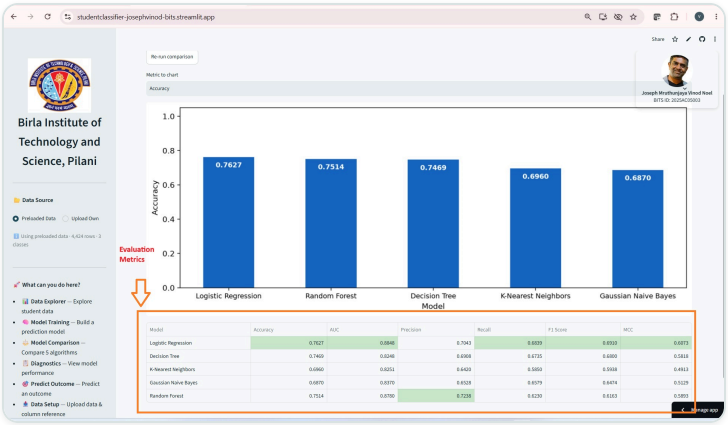



**Figure 14** — Compare Models tab: Accuracy, AUC, Precision, Recall, F1 Score and MCC shown side by side for all five models, with the best value per metric highlighted in green (Logistic Regression leading on Accuracy, AUC, Recall, F1 and MCC; Random Forest leading on Precision).

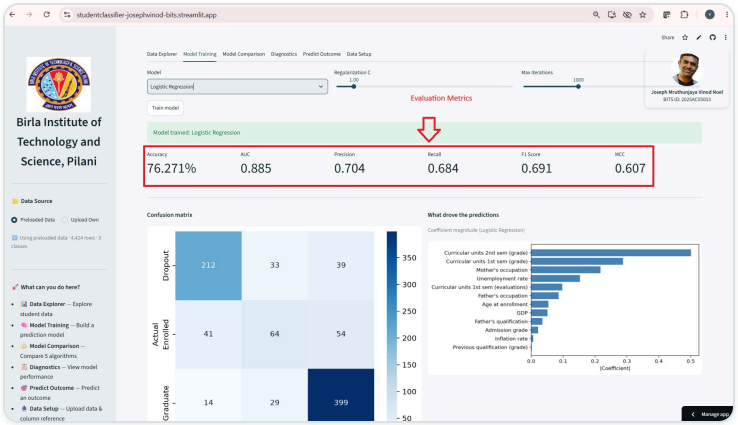



**Figure 15** — Train a Model tab: Accuracy (76.271%), AUC, Precision, Recall, F1 Score and MCC displayed for the trained Logistic Regression model.

## 5.d Confusion Matrix

Every trained model's results include a confusion matrix heatmap comparing predicted vs. actual class labels, alongside a coefficient/feature-importance chart explaining what drove the predictions.

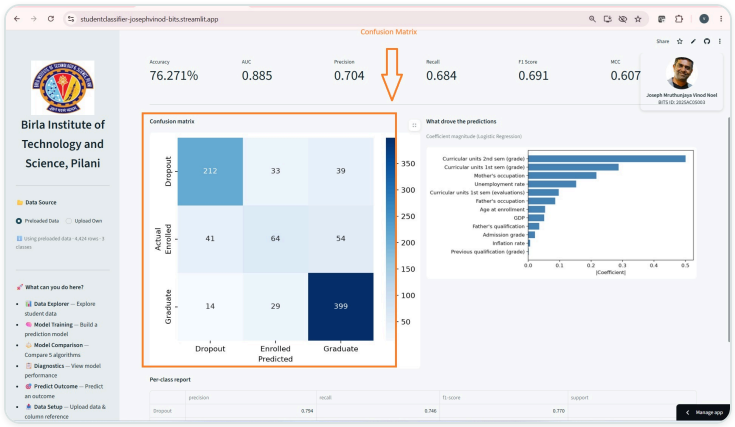



**Figure 16** — Train a Model tab: confusion matrix heatmap (Dropout / Enrolled / Graduate) for the trained model, alongside the coefficient-magnitude chart of what drove the predictions.

## 5.e Different Models on Your Test Data

The Compare Models tab retrains all five models on whichever test data is currently loaded — preloaded or uploaded — and charts their accuracy side by side, so performance across models on the same test data is directly comparable.

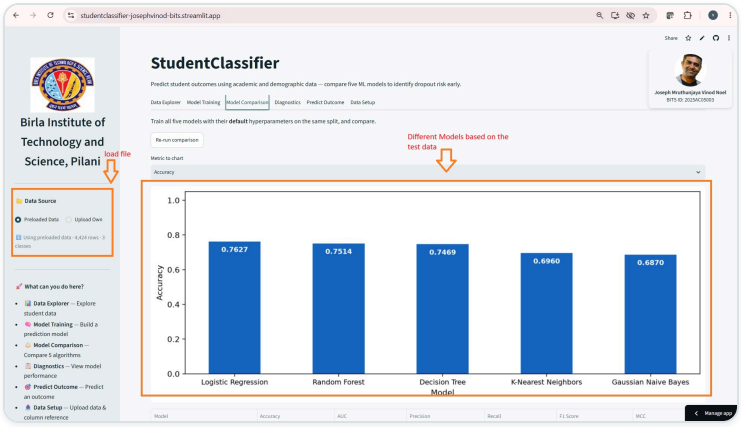


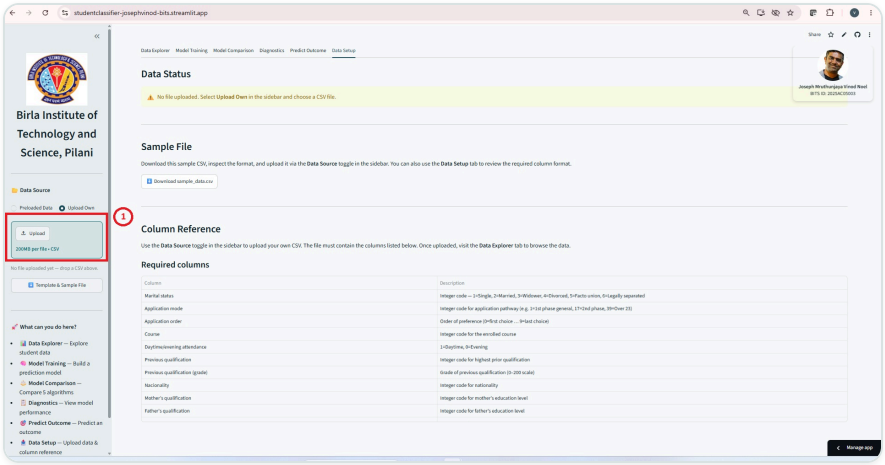

**Figure 17** — Compare Models tab: all five models' accuracy charted on the currently loaded test data, with the Data Source panel (Preloaded Data / Upload Own) visible in the sidebar.

## 6. UI File Upload Feature — Procedure and Proof

This section demonstrates, end to end, that the deployed app correctly accepts a user-uploaded CSV and uses it — instead of the preloaded dataset — throughout the rest of the app. The walkthrough below uploads a 100-row sample file and traces it through validation, data exploration, and model comparison.

### Step 1 — Click the Upload Button

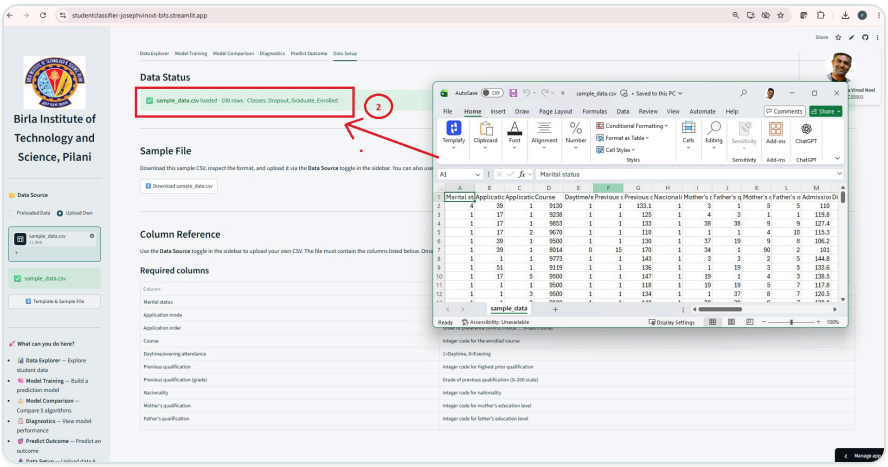
On the Data Setup tab, selecting **Upload Own** reveals the sidebar's Upload control, which accepts a CSV file up to 200MB.




**Step 1** — Data Setup tab: the sidebar **Upload** control (1) is used to select a CSV file for upload.

### Step 2 — Upload the File and Confirm Validation Status

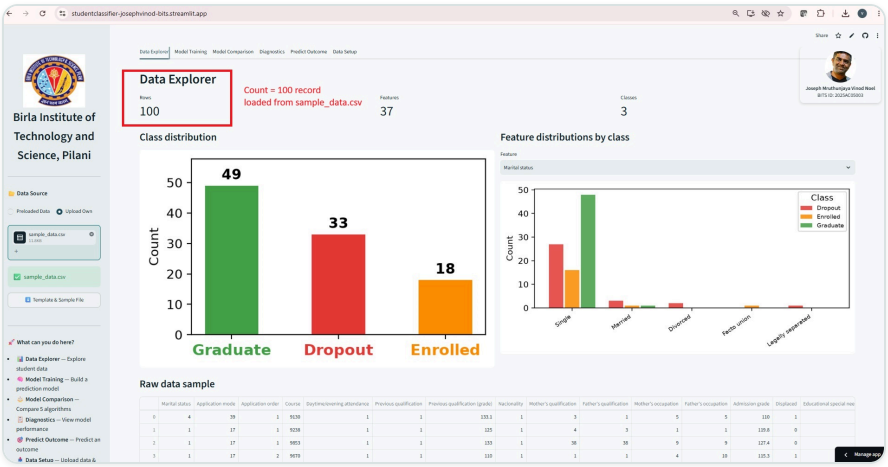
After the CSV is selected, the app validates it against the required column schema and reports the outcome in the Data Status panel.




**Step 2** — Data Setup tab: the Data Status panel (2) confirms **sample_data.csv loaded · 100 rows · Classes: Dropout, Graduate, Enrolled** once the file passes validation.

## Step 3 — Data Exploration Reflects the Uploaded 100 Records

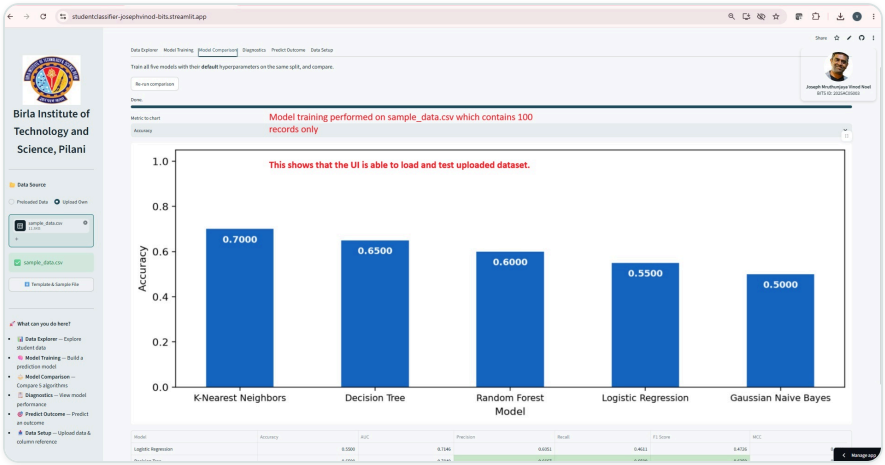
Switching to the Data Explorer tab shows the uploaded file driving every chart and table on the page.




**Step 3** — Data Explorer tab: **Rows = 100**, confirming the data explorer is reading directly from the uploaded sample_data.csv rather than the preloaded dataset.

## Step 4 — Model Comparison Reflects the Uploaded 100 Records

Finally, the Model Comparison tab retrains and compares all five models on the uploaded file, confirming the upload propagates through to model training and evaluation as well.




**Step 4** — Model Comparison tab: all five models are trained and compared on the uploaded 100-record sample_data.csv, confirming the UI correctly loads and evaluates an uploaded dataset end to end.